

Exact Paradoxical Collatz Cylinders: A Formal Search Note

Collatz proof project*

September 5, 2026

Abstract

We formalize an exact quotient-interval characterization of paradoxical shortcut Collatz segments. It permits exhaustive enumeration at each fixed length without a seed-height cutoff. Lean proves the complete length-eight classification above seed two. A separate exact-integer Python enumeration through length twenty reproduces the same five segments. A suffix-correction bound now prunes whole prefix subtrees. Tested searches at selected lengths through sixty-five complete; the length-65 census finds 244 segments. Maximal-count monotonicity and finite seed intervals reduce the search work. We also prove a limitation: any fixed finite collection of these relaxed prefix tests eventually fails to prune at large target lengths. This remains a fixed-length instrument, not a proof of global finiteness or Collatz.

1 Definition and quotient interval

Let $T(n) = n/2$ for even n and $(3n + 1)/2$ for odd n . Let $j_L(n)$ count odd steps before time L . Following Rozier and Terracol [1], a segment is paradoxical when

$$n > 0, \quad T^L(n) \geq n, \quad 3^{j_L(n)} < 2^L.$$

Intermediate descent is allowed. For a fixed residue $0 \leq r < 2^L$, set $u = T^L(r)$ and $A = 3^{j_L(r)}$. The affine transport identity is

$$T^L(r + 2^L q) = u + Aq,$$

and the odd-step count is constant on this residue class.

Theorem 1. *Assume $A < 2^L$. For every natural q , the seed $r + 2^L q$ is paradoxical exactly when*

$$0 < r + 2^L q, \quad r \leq u, \quad q \leq \left\lfloor \frac{u - r}{2^L - A} \right\rfloor.$$

Proof. Non-descent is equivalent to $r + 2^L q \leq u + Aq$, or $r + (2^L - A)q \leq u$. Since the coefficient is positive, this is precisely the stated interval and $r \leq u$. Positivity excludes the zero seed. In Lean, retaining $r \leq u$ is essential because natural subtraction truncates at zero. $\square$

To enumerate seeds at least three, impose also $q \geq \max(0, \lceil (3 - r)/2^L \rceil)$. Noncontracting residues contribute none. The unique decomposition $n = (n \bmod 2^L) + 2^L \lfloor n/2^L \rfloor$ proves coverage of all seeds at the selected length.

*Prepared with Codex (OpenAI). This is a formalization and small-case reproduction, not a literature-priority claim.

2 Checked small case and experiment

Theorem 2. *For every $n > 2$, the length-eight segment is paradoxical if and only if*

$$n \in \{7, 9, 18, 19, 25\}.$$

Proof. A kernel-decided table checks all 256 residues. Every contracting residue with $u \geq r$ satisfies $u-r < 256-A$, so its admissible quotient is zero. The possible residues are 0, 1, 2, 7, 9, 18, 19, 25; the first three are excluded by $n > 2$. The five remaining trajectories directly satisfy the definition. $\square$

Seed	Endpoint at eight	Odd steps	First descent time
7	8	5	7
9	10	5	2
18	20	5	1
19	20	5	4
25	26	5	2

The Python census exhausts lengths one through twenty, using all 2^L residues at each length and the full quotient interval. It finds only these five pairs. Each reported segment is replayed directly. Unit tests independently check every residue through length ten and interval membership and boundaries through length eight. The length-twenty census is a tested Python computation, not a Lean-extracted or kernel-certified enumeration. Only the displayed length-eight classification has been proved in Lean.

3 Research role and reproducibility

Rozier and Terracol prove that finiteness of paradoxical segments above seed two would imply Collatz [1]. Their reduction is already represented in the nested source `rwst__lean-code/RT/CRozLemma32.lean`. Its local toolchain differs from the main project's; that nested headline theorem was source-inspected, not rebuilt or axiom-audited in this pass. It is not imported by the new module. Other nearby library results use cited computational assumptions, which must not be silently imported as proofs.

The new main module is `lean/Collatz/Paradoxical.lean`. Its principal declarations are

```
paradoxical_quotient_interval
paradoxical_residue_iff
paradoxical_eight_iff
```

These are included in the expanded 124-declaration audit, with only standard foundational axioms. No new axiom, sorry, or `native_decide` is used. The table uses ordinary kernel-checked `decide`.

Build with `lake build Collatz.Paradoxical` from `lean/`. From the repository root, run `python3 analysis/paradoxical_cylinders.py`; the JSON output records the exact finite scope. Brute-force residue enumeration costs exponentially many states. The prefix pruning extension below avoids many of those states, without establishing all-length termination. The missing task is an independently proved restriction on the contracting words that could support arbitrarily long paradoxical segments. Finite experimental absence does not supply that restriction.

4 A proved prefix-pruning rule

Represent a parity word by Boolean letters in orbit order. For a word w , let $|w|$ be its length, $j(w)$ its number of ones, and $d(w)$ its affine correction. The exact composition identity is

$$d(uv) = 3^{j(v)}d(u) + 2^{|u|}d(v).$$

The maximal suffix correction at fixed length h and odd count k is bounded by

$$d(v) \leq 2^{h-k}(3^k - 2^k). \quad (1)$$

This is the same extremal correction bound appearing in Rozier–Terracol [1], Theorem 2.2, translated from the remainder $d(v)/2^h$ to the integer correction. The new module proves it directly by induction, without importing that paper’s formalization.

Theorem 3. *Let a prefix u have length s , odd count a , and correction D . Every suffix v of length h and odd count k satisfies*

$$d(uv) \leq U_k := 3^k D + 2^{s+h-k}(3^k - 2^k).$$

Suppose a seed $n \geq m$ realizes uv and $3^{a+k} < 2^{s+h}$. If

$$U_k < (2^{s+h} - 3^{a+k})m, \quad (2)$$

then its segment of length $s + h$ is not paradoxical.

Proof. The upper bound follows from composition and (1). If the endpoint is at least n , the exact affine identity gives

$$(2^{s+h} - 3^{a+k})n \leq d(uv) \leq U_k.$$

The positive coefficient and $n \geq m$ contradict (2). □

For a canonical prefix residue $r \bmod 2^s$, the smallest seed at least three in the class is

$$m = r + 2^s \max\left(0, \left\lceil \frac{3 - r}{2^s} \right\rceil\right).$$

Let k_* be the largest $k \leq h$ for which $3^{a+k} < 2^L$, where $L = s + h$. If no such count exists, there is no paradoxical completion. Otherwise it is sufficient to test k_* alone. Lean now proves the required monotonicity: for $k \leq l \leq L$,

$$3^k D + 2^{L-k}(3^k - 2^k) \leq 3^l D + 2^{L-l}(3^l - 2^l).$$

To see this, add 2^L to the suffix-only term to obtain $2^{L-k}3^k$. Increasing k replaces factors of two by factors of three. The prefix contribution $3^k D$ also increases, while $(2^L - 3^{a+k})m$ decreases. Thus passing the necessary test at any count implies passing at the maximal admissible count.

Resolving finite seed intervals before reaching a leaf

The bound applies to every realized seed n , not only the smallest seed m of a prefix class. Every paradoxical completion satisfies

$$(2^L - 3^{a+k_*})n \leq U_{k_*}, \quad U_k = 3^k D + 2^{L-k}(3^k - 2^k).$$

Consequently a class $n = r + 2^s q$ has a finite possible quotient interval already at depth s . The implementation replays the candidates directly when at most sixteen remain. Each such node is then resolved completely; its children need not be explored. A candidate is accepted only after its actual endpoint and odd count satisfy the definition. This finite bound is proved as `paradoxical_completion_seed_bound`.

Exact eight-step residue tables accelerate the replays using the same affine transport identity as Section 1. Every emitted segment is also replayed one step at a time. The search implementation remains Python, not a kernel-certified traversal.

The induction behind (1) first proves the division-free inequality

$$2^{j(w)}(d(w) + 2^{|w|}) \leq 2^{|w|}3^{j(w)}.$$

Then $j(w) \leq |w|$ and cancellation of $2^{j(w)} > 0$ give (1). These steps and the exclusion theorem are checked in `Collatz.ParadoxicalPruning`.

5 Search results and verification boundary

The implementation `analysis/paradoxical_pruned.py` follows residue lifts in a depth-first tree. Each node stores its depth, residue, endpoint, odd count, and correction. The affine identity is checked at every visited node. Small candidate intervals are resolved directly; any remaining leaves use Section 1. There is no arbitrary seed-height cutoff.

Length	Tree nodes	Direct seed checks	Segments	Status
8	13	80	5	Complete
20	23	133	0	Complete
27	955	5,159	50	Complete
32	703	3,699	0	Complete
40	2,889	15,703	0	Complete
65	5,324,915	27,386,515	244	Complete

These are selected lengths, not every intervening length. Tree nodes and direct candidate checks are distinct work units; both are reported. The length-65 run finishes with zero pending nodes, replacing the previous incomplete two-million-node result. Its 244 seeds range from 73 to 4547, all have 41 odd steps, and all had descended below their seed before the endpoint. A full residue scan at that length has 2^{65} classes.

The ten Python controls include exhaustive comparisons through length fourteen, comparison with the old all-count test, direct-resolution enabled and disabled, exact jumps compared with individual steps, and incomplete-run status checks. A separate direct scan of seeds below 65,536 at length 65 reproduces all 244 reported seeds. This scan verifies that finite slice, not absence above its cutoff; the all-seed coverage relies on the pruned traversal and its bounds. The JSON file `analysis/paradoxical_pruned_results.json` preserves the results.

Formal and computational claims

The module proves the correction bound, maximal-count monotonicity, finite seed bound, and pruning implications. These mathematical statements are in Lean. The Python tree traversal and the reported larger-length counts are tested computations, not Lean-extracted code or kernel-verified search certificates. The existing all-seed length-eight classification remains separately kernel-proved. The selected 124-declaration audit includes the new bounds and reports only standard foundational axioms.

Build from `lean/` with

```
lake build Collatz.ParadoxicalPruning
```

From the repository root, reproduce the experiment with

```
python3 analysis/paradoxical_pruned.py --work-limit 20000000
```

The result makes fixed-length investigation substantially more practical, but does not yield a uniform cutoff on possible lengths. The earlier length-65 work limit has been overcome computationally. The limitation below shows why these envelopes alone do not yield closure with a fixed finite prefix tree. A restriction uniform in length or circuit count is still needed.

6 A limitation of the relaxed envelope

There is also a precise mathematical limitation of the pruning test itself. Keep a prefix length s , odd count a , correction D , and seed floor m fixed. The test relaxes the unfinished word to an arbitrary arrangement with a chosen odd count k .

Theorem 4. *For any natural s, a, D, m and every target length*

$$L \geq s + 2(a + 2m) + 1,$$

there exists $k \leq L - s$ such that

$$3^{a+k} < 2^L, \quad (2^L - 3^{a+k})m \leq 3^k D + 2^{L-k}(3^k - 2^k).$$

Thus at least one contracting odd-count choice fails the strict pruning inequality.

Proof. Choose $k = 2m$. Elementary induction gives

$$(m + 1)2^{2m} \leq 3^{2m}.$$

For the inductive step, multiply the previous bound by nine and use $4(m + 2) \leq 9(m + 1)$. Hence $3^k - 2^k \geq m2^k$. The length threshold ensures both $k \leq L - s$ and

$$3^{a+k} \leq 4^{a+k} = 2^{2(a+k)} < 2^L.$$

Finally,

$$3^k D + 2^{L-k}(3^k - 2^k) \geq 2^{L-k} m 2^k = 2^L m \geq (2^L - 3^{a+k})m.$$

□

For a finite collection of fixed prefix records, take the maximum of their thresholds. All records then simultaneously survive the relaxed test at every larger length. This consequence is also proved in Lean.

Scope and strategic consequence

Surviving the envelope is only a necessary test: the theorem constructs an odd-count budget, not a realizable parity suffix, integer trajectory, or paradoxical segment. It does not invalidate pruning at a specified length, nor exclude proofs using target-dependent depth, stronger arithmetic, or another method. It rules out closing all sufficiently large lengths with a fixed finite collection of these relaxed envelope tests alone.

The envelope discards compatibility between a suffix’s correction and its realizing seed residue. The next argument must retain this arithmetic or supply a separate uniform restriction.

Source: `lean/Collatz/PruningLimit.lean`. The explicit threshold and finite-family declarations are

```
completion_envelope_survives  
finite_envelopes_eventually_survive.
```

Both declarations pass the 124-theorem axiom audit. Python checks the thresholds on every residue prefix through depth seven. Collatz remains open.

References

- [1] O. Rozier and C. Terracol, *Paradoxical behavior in Collatz sequences*, arXiv:2502.00948v3 (2025), Definition 1.1 and Section 3. <https://arxiv.org/html/2502.00948v3>.